Transbase
Release Notes

Transaction Software GmbH
Willy-Brandt-Allee 2
D-81829 München
Germany
Phone: +49-89-62709-0
Fax: +49-89-62709-11
Email: info@transaction.de
http://www.transaction.de

Version 5.3
July 07, 2005

Contents

Chapter 1

Version 5.3.1.42

1.1 Database Encryption

Transbase now supports database encryption. Database encryption must be enabled by using `tbadmin`.

Chapter 2

Version 5.3.1.41

2.1 Tbadmin Library

All API's in tbadmin library have been defined to STDCALL. This makes calling the API from VB or similar SDK's more easier.

Chapter 3

Version 5.3.1.36

3.1 Evaluationplan Support

Support for Evalutation plans can by enabled by a `tbmode`-statement. Support is enabled by `tbmode plans on` and disabled by `tbmode plans off`. Plans can be retrieved on `tbx`-level by calling `tbx (TB__GETPLAN, ...)`. `tbi` supports `Evaluation Plans`

3.2 Administration Password

An administration password has been introduced for creating, deleting and administrating databases. As a consequence `tbadmin`-password is not longer needed for deleting a database.

3.3 Library support for Backup Utilities

3.3.1 TBTar Library

A library is supplied for developing tbtar compliant applications. A more detailed description is given in TBTar SDK

3.3.2 TBArc Library

A library is supplied for developing tbarc compliant applications. A more detailed description is given in TBArc SDK

Chapter 4

Transbase SQL extensions

4.1 SEQUENCE

A new construct `SEQUENCE` has been introduced which serves to generate databasewide unique numbers. A sequence is constructed with a user defined name. A start value and an increment value can optionally be specified. Per default a sequence starts with value 1 and increments by 1. A sequence with name S provides two functions, namely `S.nextval` to generate and deliver a new number and `S.currval` to refer to the most recently delivered number within the current application.

An existing `SEQUENCE` can be used as part of a `DEFAULT` clause for a field of a table. In this way, a field serves as an automatically generated synthetic key.

Example:

```
CREATE SEQUENCE si START 0;
CREATE SEQUENCE sii START 0;

CREATE TABLE invoice (
  n INTEGER DEFAULT si.NEXTVAL,
  custid DECIMAL(5),
) KEY IS n

CREATE TABLE invoice_item(
  n INTEGER REFERENCES invoice(N),
  item INTEGER DEFAULT sii.NEXTVAL,
  description VARCHAR(80),
  price DECIMAL(12,2),
  ...
) KEY IS n, item;
```

The following statements could be used to generate a new invoice with two items.

```
INSERT INTO invoice (n, custid)
VALUES (si.NEXTVAL, 12345);

INSERT INTO invoice_item(n, item, description, price)
VALUES (si.CURRVAL, sii.NEXTVAL, 'First', 33.33);

INSERT INTO invoice_item(n, item, description, price)
VALUES (si.CURRVAL, sii.NEXTVAL, 'Second', 123.45);
```

4.2 Secondary Indexes

In secondary indexes, participating fields and secondary keys now can be specified separately.

For example, assume a table with fields f1, f2, f3, f4 where f1 is the key:

```
CREATE TABLE f (
  f1 INTEGER,
  f2 INTEGER,
  f3 INTEGER,
  f4 INTEGER)
  KEY IS f1 ;
```

Analogously, it is now possible to specify a `KEY` clause for secondary indexes that implies an automatic uniqueness restriction. So, the following two notations are equivalent:

```
CREATE UNIQUE INDEX findex ON f ( f2, f3, f4)

CREATE INDEX findex ON f ( f2, f3, f4) KEY IS f2, f3, f4
```

The semantic separation between index fields and unique-key fields, however, allows constructs like:

```
CREATE INDEX findex ON f ( f2, f3, f4) KEY IS f2, f3
```

where the uniqueness restriction is given by (f2, f3) only. Although a UNIQUE INDEX on (f2, f3) could have been defined, it may be very useful to have additional fields in a secondary index in order to omit the base tuple materialization. The query

```
SELECT f2, f4 FROM f WHERE f2=5
```

would need to access the base tuples when an index is defined only on (f2, f3) while it could be resolved solely from the secondary index when an index contains (f2, f3, f4).

4.3 HyperCube Indexes

For Transbase HyperCube, secondary indexes can be built on the numeric data types: `TINYINT`, `SMALLINT`, `INTEGER` and `NUMERIC(P,S)`.

Example:

```
CREATE TABLE points (
  id INTEGER,
  x NUMERIC (10,7) NOT NULL CHECK(x BETWEEN -180 AND 180),
  y NUMERIC (10,7) NOT NULL CHECK(x BETWEEN -90 AND 90),
  info CHAR(*),
  ... )
KEY IS id ;

CREATE INDEX xpoints ON points(x,y,info) HCKEY IS x,y ;
```

The secondary index contains the fields x, y,and info. The hypercube key is defined on x and y only. Note that this example uses the new extended syntax for secondary indexes described above.

All fields used as hypercube key in the secondary index must be of appropriate type and be specified as `NOT NULL` and have a `CHECK CONSTRAINT` with range condition in the undelying table definition.

4.4 Tuple Construction

For SQL2 conformity, tuples can be constructed by parentheses as well as by brackets, e.g.

```
where (tuple) in (select ...).
```

4.5 ORACLE Compatibility

Various ORACLE functions have been resembled with identical semantics and syntax, including `SYSDATE`, `DECODE`, `TO_CHAR`, `NVL`, `TRUNC` in order to alleviate the migration from ORACLE to Transbase.

4.6 Optimizations

Various optimizations improve the computation of predicates on B-tree layer. Projections are handeled by the B-tree layer, too, in order to save space and time of processing.

Space requirements of operator trees have been reduced for joins.

Chapter 5

UNICODE Databases

The UNICODE standard maps characters of all existing languages into a single coding schema. Characters are either denoted by 16-bit values (UCS-2) or by 32-bit values (UCS-4). For databases, however, to store each character as fixed 2 or even 4 bytes, would be too inefficient. Instead, Transbase uses the UTF-8 coding of UNICODE characters to store them on disk and to send and receive them over communication lines.

The table below shows valid UTF-8 codings:

```
0xxxxxxx                                                    7 bits
110xxxxx  10xxxxxx                                         11 bits
1110xxxx  10xxxxxx  10xxxxxx                               16 bits
11110xxx  10xxxxxx  10xxxxxx  10xxxxxx                     21 bits
111110xx  10xxxxxx  10xxxxxx  10xxxxxx  10xxxxxx           26 bits
1111110x  10xxxxxx  10xxxxxx  10xxxxxx  10xxxxxx  10xxxxxx 31 bits
```

In particular, it can be seen that

- ASCII characters in the range of 0..127 are mapped into themselves;
- single-byte characters in the range 128..255 (i.e. those in the Latin-1 charset) are mapped into a sequence of two bytes;
- other UCS-2 characters are mapped into sequences of at most three bytes;
- UCS-4 characters are mapped into sequences of at most six bytes.

It can be seen, too, that

- not every sequence of 8-bit bytes is a valid UTF-8 coded string,

- the mapping is only unique if characters are mapped into sequences of minimal length and
- the sort order is preserved by the coding.

The fact that a single character may consist of more than one byte, influences the string functions and string predicates. The `SUBSTRING` function e.g. assumes its parameters to be characters (not bytes) and the `LIKE` predicate expects an underscore to denote a single character (not byte).

Please note that the definition of fixed length characters like CHAR(N) still is given in bytes (not in characters) because a length of 2 characters would result in a (non-fixed) bytelength of between 2 and 12. A length restriction in characters could be formulated as: `CONSTRAINT (CHARACTER_LENGTH(x) = 2)`.

At the communication layer, UTF-8 is used, too. In particular, TBX strings are assumed to be valid UTF-8, both on input (queries, parameters) and on output (result strings). UTF-8 should be mapped into either UNICODE or into an appropriate single-byte charset by the application.

For conversion of single-byte databases, it is recommended to export the data, convert it into UTF-8 or UNICODE and then to import it. The Transbase Spooler has been extended to accept single-byte, UTF-8 and UNICODE input.

In particular, the following important modifications apply for Unicode databases.

5.1 tbadmin

The coding of databases has to be specified upon creation. It cannot be changed later. Code pages can be: proprietary (upward-compatibel to former Transbase versions), ASCII (only ASCII characters below 128 are permitted), single-byte (arbitrary single-byte characters are permitted), and UTF-8 (arbitrary Unicode characters are permitted).

Along with the coding, a fixed locale setting can be defined which influences national character processing.

5.2 SQL

5.2.1 CHAR(N), VARCHAR(N), SIZE OF

When defining fixed-length strings, a maximum number of bytes has to be specified rather than a maximum number of characters. The reason for this decision is that the specification of N is space-related. In particular, CHAR(5) strings must have

fixed length of 5 bytes, while strings of five characters may vary in size between 5 and 15 bytes(for UCS-2).

A size restriction for a field COL in terms of characters can be specified by an additional `CHECK CONSTRAINT (CHARACTER_LENGTH(COL) < 10)`.

In contrast, the SIZE OF operator for strings returns the number of bytes taken by the UTF-8 representation.

5.2.2 String functions

All string related functions work in terms of characters (not bytes).

In particular, the function `CHARACTER_LENGTH(x)` returns the number of Unicode characters in x, while `SIZE OF x` returns the number of bytes.

The function `SUBSTRING(x from 1 for 2)` returns the first two characters of x. `SIZE OF (SUBSTRING(x from 1 for 2))` may return values between 0 and 6 for UCS-2 strings.

5.2.3 String literals

String literals containing UNICODE characters can be constructed by either of the following methods:

```
'Mnchen'

'M' + 0u00fc + 'nchen'

'M' + 0xc3bc + 'nchen'
```

The first method requires the ability to type the unicode character directly (which be not be possible in any environment).

The second method uses a new string literal variant which denote Unicode characters by their hexadecimal value. Note that always four hexadecimal digits must be specified per Unicode character. Thereby, more than one Unicode character representation can be concatenated, e.g. 0u00fc006e.

The third method provides a direct UTF-8 representation (syntactically a BINCHAR) of that part of the string that cannot be expressed literally. Again note, that for this representation a valid UTF-8 coding is required.

5.2.4 Spool files

When data is loaded from or extracted into external files by the Transbase SPOOL command, it has to be specified in which format the file should be coded.

For this purpose, the syntax of the SPOOL statement has been extended. See SQL guide for more details.

Chapter 6

Incremental ROM files for TransbaseCD

TransbaseCD offers a new space saving possibility to propagate database updates for CD-ROM databases.

To propagate updates onto a CD-ROM database (which might be distributed in many copies) two alternatives have been offered in the past:

1. Distribute SQL scripts (and possibly spool files) which are executed on CD-ROM site.
2. Accumulate all updates on one master CD-ROM database (updates then are reflected in a corresponding diskfile). Then distribute copies of the diskfile to the target sites.

The first solution is complex and requires runtime equipment on the target sites. The second solution is platform dependent and tends to produce large data volumes.

A third method is provided now: Updates are prepared as in the second solution on one master site. Then a new kind of FLUSH operation ("incremental flush") is performed on that CD-ROM database. Thereby, so-called delta romfiles are produced. These delta romfiles then are distributed to the target sites. Blocks appearing in a delta romfile shadow the blocks in the original romfiles. Delta romfiles are automatically compressed and thus are ideally suited to be distributed over the Internet. Moreover, delta romfiles become platform independent, too, like standard romfiles.

Chapter 7

Administration

7.1 Case Insensitive Databases

Databases can be defined to be "case insensitive". In this case, all identifiers (field names, table names etc.) are compared by case-insensitive routines. When inserted into the data dictionary, those identifiers are mapped to capitals. This has to be considered when the data dictionary is searched explicitly by SQL queries on system tables.

Note that "delimiter identifiers" are still case sensitive.

By default, databases are case sensitive. Migrated old databases are also case sensitive. A case sensitive database can also be converted to a case insensitive database by an explicit `tbadmin -a` command. In rare cases, this conversion might fail if table or field names exist which would map to the same capital letter identifier.

7.2 TBADMIN functions

The cache size (global shared memory) of a database now can be set up to a maximum value of 20 Gbyte.

Temporary files possibly needed by the kernel are no longer placed in the TMP or TMPDIR directory, but in the scratch directory of the database. Therefore, it should be made sure that sufficient space is available for scratch files.

7.3 TBMUX: Crypted Communication

The communication between Transbase kernel and applications is handeled by TBX. The communication can be declared to be crypted in order to increase the

privacy of the client/server communication.

Cryptification is specified by the command `tbmux -crypt ...` and therefore applies to all communications with this database server.